Open in app ↗ Sign up Sign in

Medium Search Write

Workflow Orchestration 101: Why Your Services Need a Conductor

Michael Ortiz Follow 7 min read · Feb 22, 2025

2

Imagine an orchestra — musicians playing together, led by a conductor. In software, that's orchestration: a way to keep your services in sync. Let's dive into why your system needs a conductor to scale in the world of microservices and serverless architectures.

What is Orchestration 🤔

In ***music***, an *orchestra* is group of musicians who play different instruments together. Musicians are led by a ***conductor*** who is also known as the artistic leader of the orchestra:

The Conductor: Leader of the Orchestra

The main responsibility of a *conductor* in music is to lead and coordinate the orchestra by keeping the tempo, signaling groups of musicians to decrease or increase intensity and volume of a performance amongst other things.

What is Orchestration (in the context of Software)

Now that we know the basics of the concept in the context of music, let's translate this into the context of **Software Services**:

> *"Orchestration is the the coordination and management of multiple computer systems, applications and/or services, stringing together multiple tasks in order to execute a larger workflow or process" — From the Web*

Wait a sec — let's break it down. Just like a musical orchestra, software orchestration follows the same idea: a workflow acts as the sheet music, the

orchestrator plays the conductor, and services are the musicians bringing it all to life. Let's illustrate this below:

Orchestrator Analogy Illustration

Let's Dive In 🤿

In software, as applications scale and become larger and more complex, it becomes critical to keep services decoupled to maintain flexibility and resilience.

Traditionally, software was built as **monoliths** — large applications or services that run within a singular codebase. While this approach can work in the beginning , it becomes difficult to manage as software requirements change and scale. The main reason to this is because business logic and functionality are tightly coupled within one massive application. This means that if a critical issue, such as a memory leak, occurs, it can potentially bring down the entire system.

A common solution to this problem is to break down a monolithic application into **smaller, independent** services known as *microservices* or even *serverless functions*.

These services are designed to follow a Single Responsibility Principle which in another words says: "*Each service should have one, and only one, well-defined business function or responsibility, essentially doing one thing and doing it well.*"

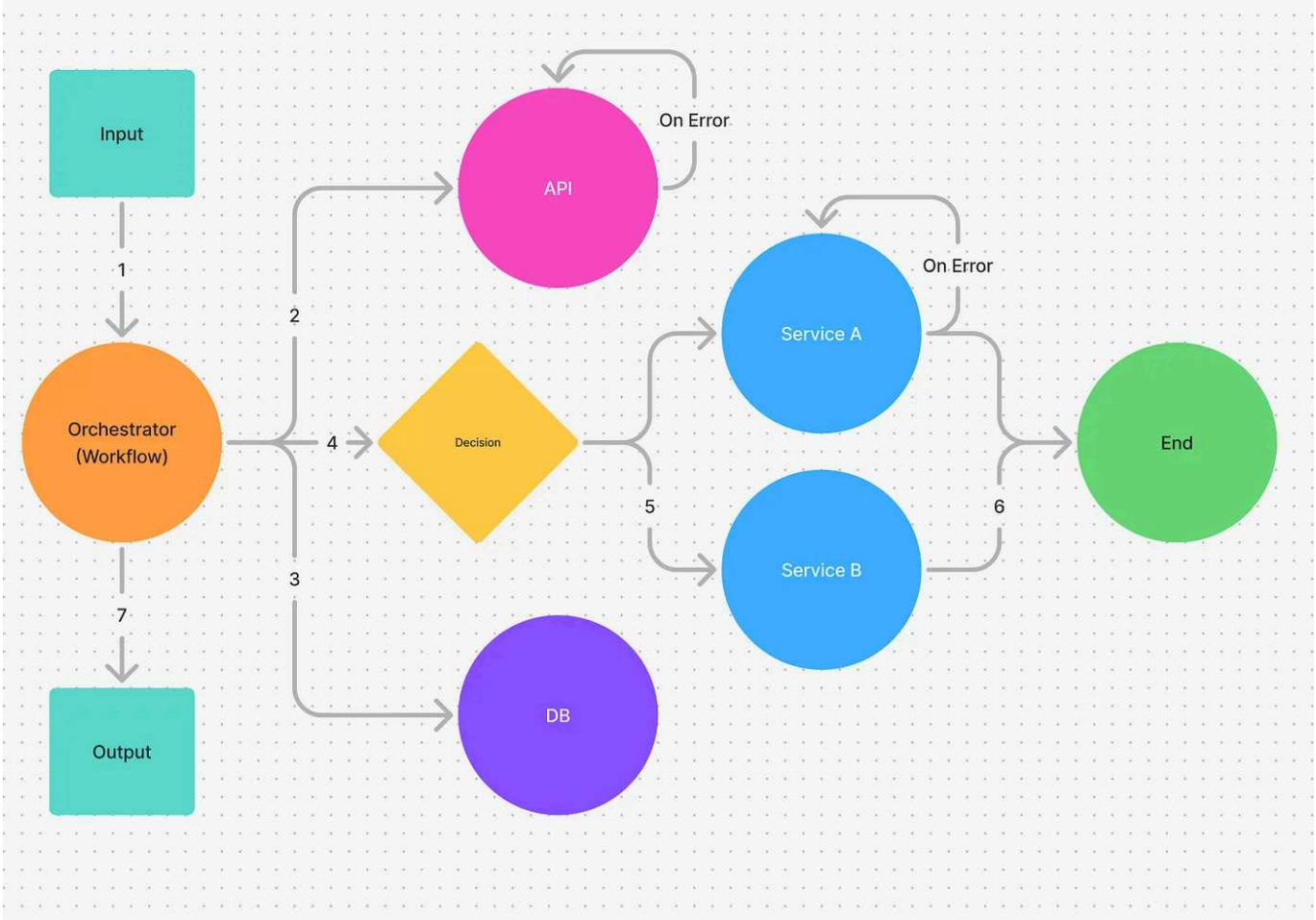
This approach improves scalability, as these smaller services collectively form a larger system without a single point of failure.

Workflow Orchestration 🚀

Great! Now that we understand why these large systems are broken down into smaller units, we need a way to coordinate (*or conduct*) how these services are called step by step.

By using a defined workflow (usually defined in JSON format), we can define how each step calls a service, and ultimately provide an expected output when the workflow is completed.

Here is a visual representation of workflow orchestration:



Workflow Orchestration simplifies the coordination of services (by using steps), enabling a system to scale efficiently while keeping each service independent and decoupled from others. By providing an abstraction layer, this ensures that tasks and business logic are executed as intended. It manages input, output, error handling, and retries at each step of the way, ensuring your process remain stable and consistent.

Here are some benefits of workflow orchestration:

1. **Loose Coupling of Services**
   Orchestrators removes the need for services to depend directly on one another, enabling them to be coordinated externally through a workflow definition.

2. **Independent Execution of Tasks**
   Tasks or steps operate in isolation, running on their own terms while the orchestrator handles their sequence and coordination.

3. **Resilience & Fault Tolerance**
   It provides built-in error handling, retries, and rollback capabilities, workflows stay resilient which keeps the system afloat even when failures occur.

4. **Observability & Monitoring**
   Detailed logs, traces, and metrics offer a clear view of workflow performance, making it easy to spot and fix issues quickly.

Types of Workflows 🏋️

Depending on the framework you use, workflows generally fall into two main categories:

Asynchronous Workflows
These workflows are ideal for long-running processes that operate in the background. They don't require an immediate response, and the output becomes available once the workflow has finished processing.

Synchronous Workflows
Designed for short-lived, quick-response tasks, synchronous workflows are ideal when the user expects a prompt output. These workflows handle tasks that need to be executed rapidly and return a result without delay.

From these two types of workflows, **asynchronous workflows** are more commonly used for long running jobs, followed by synchronous workflows.

In either case, depending on your expected response times, one can be over the other.

🧪 Use Cases

A great example of an *asynchronous* workflow is **order processing** for an *e-commerce* website.

Get Michael Ortiz's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email Subscribe

When an order is placed, several steps need to happen in the following order:

1. Accepting payment
2. Updating inventory
3. Generating the order
4. Sending a confirmation email,

Lastly, all steps should handle any errors gracefully at every step *(as we most complete the order at all cost)*

These tasks may take a few seconds, but by orchestrating them through an asynchronous workflow, we can manage and scale these steps effectively, ensuring a smooth process that can accommodate different outcomes.

On the other hand, **synchronous workflows** are best used for simple tasks that require a fast response at the moment of the initial request. These workflows can delegate small tasks, such as calling multiple services in sequence, and return a unified output almost instantly.

Ultimately, workflow orchestration tool can significantly improve the management and execution of tasks and processes in software systems, by keeping them decoupled, and providing a framework of error handling and reliability without the need to handle this directly in code.

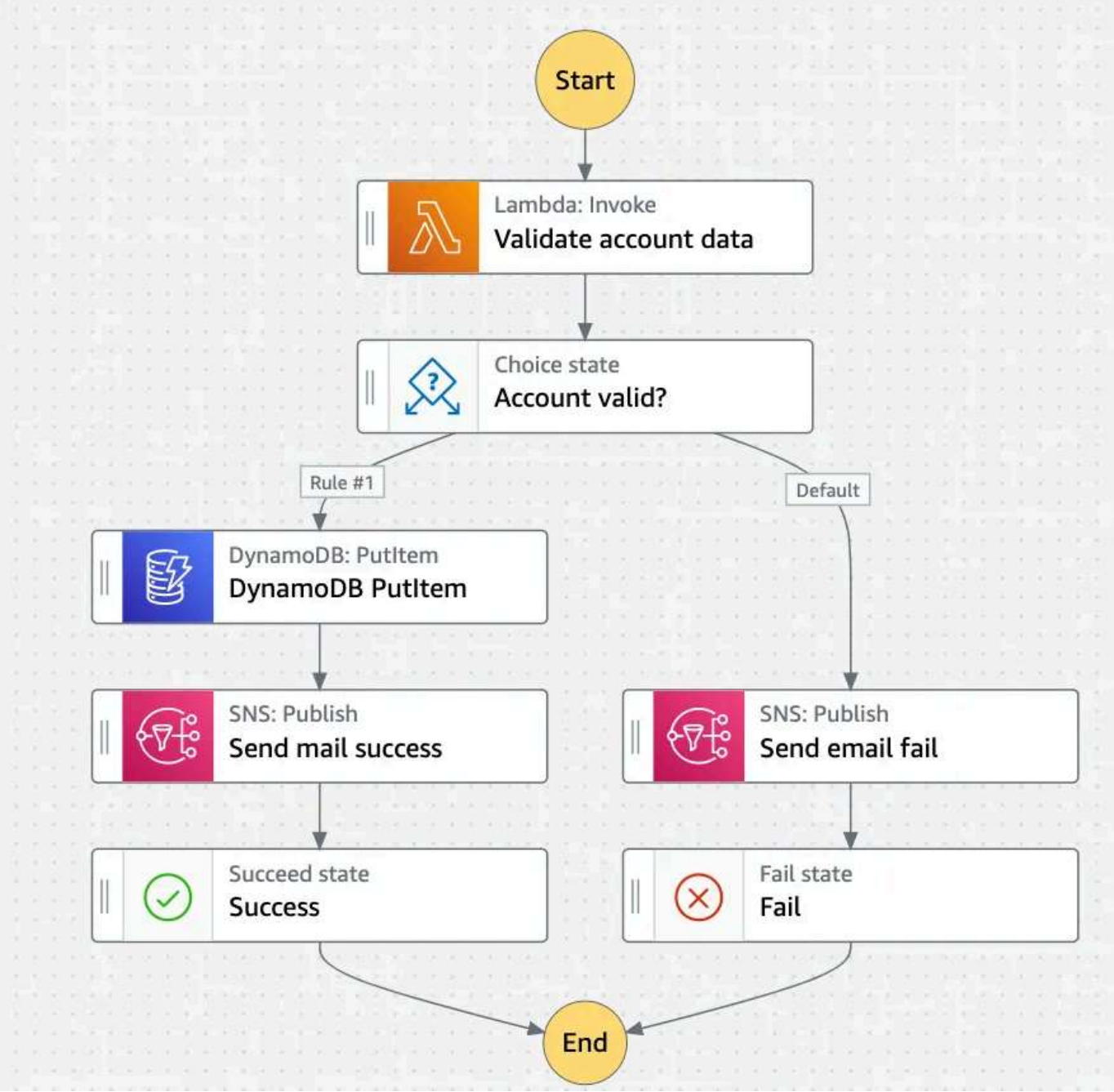
🛠️ Workflow Orchestrator Tools

Below are some of the most common workflow orchestrator tools:

AWS Step Functions

> *Step Functions is a visual workflow service that helps developers use AWS services to build distributed applications, automate processes, orchestrate microservices, and create data and machine learning (ML) pipelines.*

If you're leveraging AWS and extensively using serverless services, Step Functions is an excellent tool that integrates seamlessly with other AWS services natively. You can define workflows using Amazon States Language (JSONata) or through the intuitive AWS Console Step Functions UI Editor.

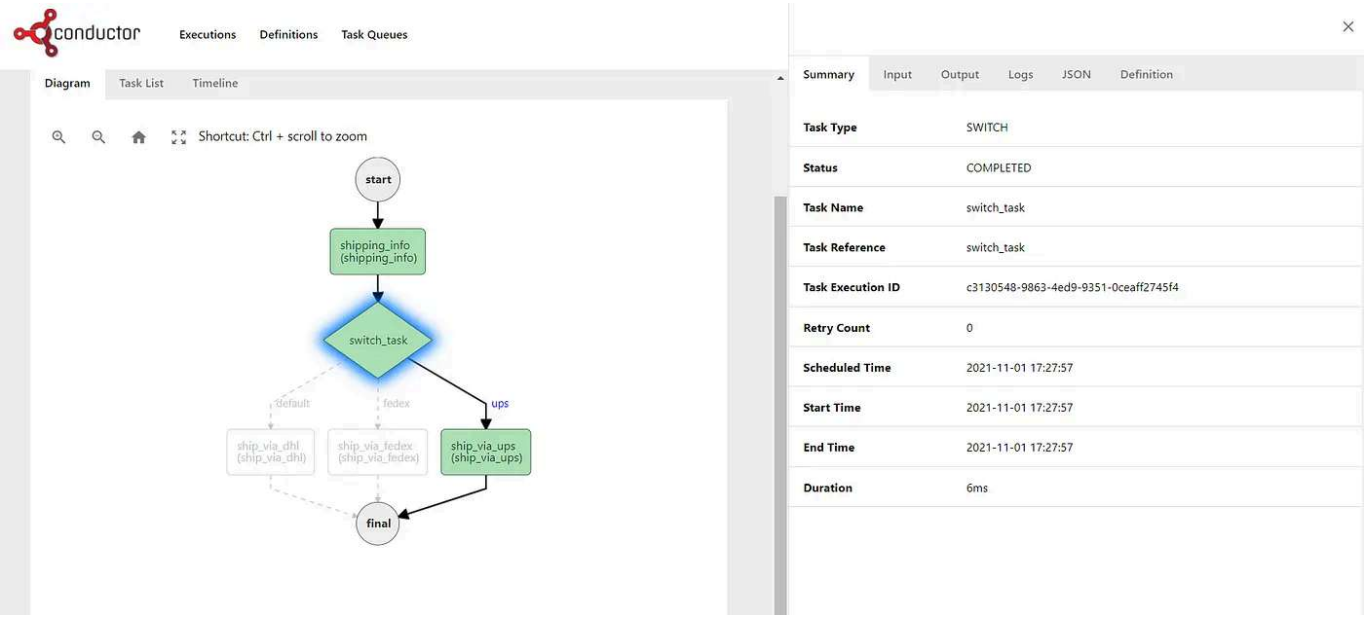


An example Step Functions Workflow

Conductor OSS (Netflix)

> *Conductor is a platform originally created at Netflix to orchestrate microservices and events. Conductor OSS is maintained by the team of developers at Orkes.io along with members of the open source community.*

Conductor is an open-source orchestration platform built to streamline task coordination across microservices. Initially created by Netflix, it enables the definition of complex workflows using JSON, allowing users to trigger APIs directly or leverage task workers that poll from a queue to execute specific steps. It includes both a user interface for workflow visualization and management, as well as API servers for integration and automation.



An example Conductor Workflow

🤔 Which Workflow Orchestrator Should I Use?

This is great question, and ultimately the decision depends on a number of factors and what your requirements are:

Step Functions

AWS Step Functions is an ideal choice for fully AWS-based distributed systems, especially when leveraging serverless services like AWS Lambda. It integrates seamlessly with a wide range of AWS services, simplifying management and reducing complexity. As a serverless service itself, Step Functions eliminates the need to manage servers — scaling automatically based on demand.

Here is my guide on Step Functions on Medium.

Conductor OSS

One of the key benefits of using Conductor OSS is its cloud-agnostic nature, allowing it to run anywhere. It's an ideal solution for distributed systems primarily built around microservices and APIs. Because Conductor OSS is an open source solution, you will need to host and maintain the servers yourself. If you prefer to not manage the servers, you can alternatively use **Orkes** which is a managed Orchestration Platform, built on Conductor OSS.

In short, AWS Step Functions is great for quick, less complex setups with less overhead, while Conductor OSS is a better fit when you need deep customization and control over your workflows, especially in microservice-heavy environments. The best way to get a sense of which one is best for you, is to get familiar with both of these tools, and experiment on your own. I will be writing some guides on using both, so stay tuned!

Challenges to Watch For

Orchestration isn't perfect. Poorly designed workflows can feel like herding cats, adding complexity to manage. Latency between services or an overloaded orchestrator can drag things down too. With asynchronous workflows, your app must handle task completion queries and process the results. For instance, if a website queries multiple slow-responding services, you might need to poll the workflow status or use web sockets to alert the user when it's done processing.

Final Thoughts

Workflow orchestration offers significant advantages. Whether you choose to implement an orchestration system or opt for event-driven architectures,

which also provide decoupling benefits, it's important to carefully evaluate your use case and choose the approach that makes the most sense.

For smaller applications, simplicity often works best. However, always consider your future growth plans. If you anticipate scaling and require a system that involves multiple steps to complete a process, or require a very fault tolerant solution that handles different outcomes, delegating these steps to a workflow orchestration system can be an ideal choice.

Ultimately, a workflow provides a visual process representation of the work being coordinated. This alone is extremely very valuable!

Let me know in the comments below if this was helpful and if you plan on trying either of these tools for your workloads 🤘 .

Thanks for reading!

Workflow Orchestration | Netflix Conductor | Aws Step Functions | Microservices | AWS

Written by Michael Ortiz

64 followers · 7 following

I write AWS and Tech content, with the goal of making concepts easy to understand by anyone one topics that are relevant in the cloud space.

Follow

No responses yet

Write a response

What are your thoughts?

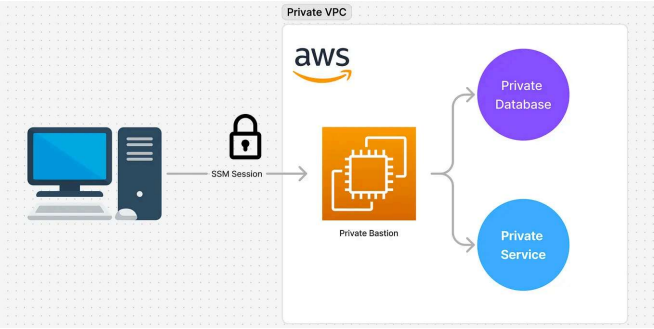
More from Michael Ortiz

Michael Ortiz

AWS: Lambda, ECS, or EC2—Which One Should You Choose?

If you're trying to choose between AWS Lambdas, ECS, or EC2 for your project, you'r...

Mar 2, 2025 1

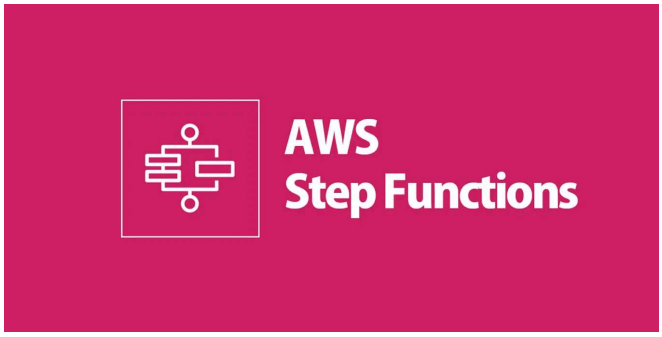


In AWS Tip by Michael Ortiz

AWS: Don't SSH into Your Public Bastion — Use SSM Session...

Using a Public EC2 Bastion to access your VPC resources through SSH from your local...

Feb 24, 2025 16 2

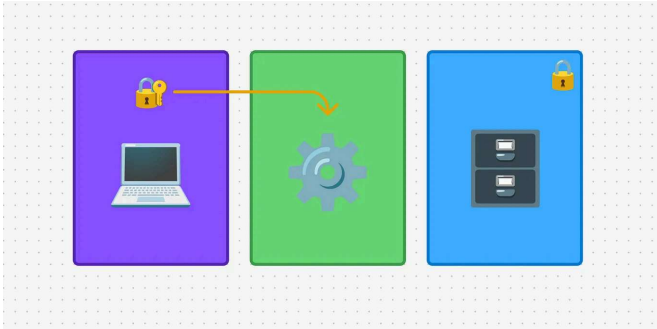


In AWS Tip by Michael Ortiz

AWS: A Step Functions Workflow Orchestration Guide

In this guide, I'll walk you through the implementation of a Step Functions Workflo...

Feb 23, 2025 20



In AWS Tip by Michael Ortiz

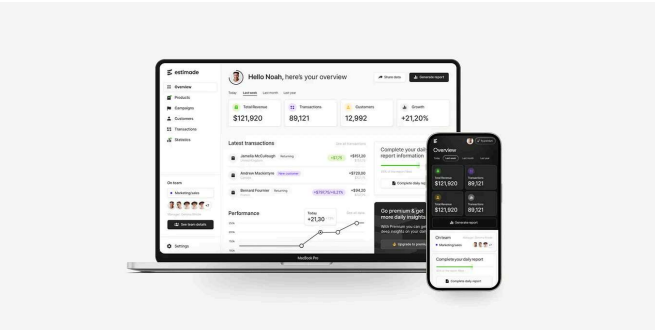
AWS: Building a Secure Modern Web Application Architecture

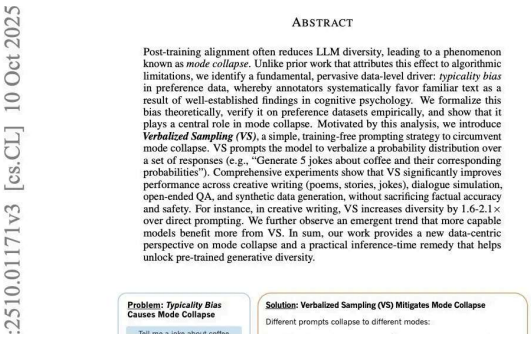
Web development has come a long way, from the days of basic static pages to the dynami...

Mar 22, 2025 15

See all from Michael Ortiz

Recommended from Medium





:2510.01171v3 [cs.CL] 10 Oct 2025

ABSTRACT

Post-training alignment often reduces LLM diversity, leading to a phenomenon known as *mode collapse*. Unlike prior work that attributes this effect to algorithmic limitations, we identify a fundamental, pervasive data-level driver: *typicality bias* in preference data, whereby annotators systematically favor familiar text as a result of well-established findings in cognitive psychology. We formalize this bias theoretically, verify it on preference datasets empirically, and show that it plays a central role in mode collapse. Motivated by this analysis, we introduce ***Verbalized Sampling (VS)***, a simple, training-free prompting strategy to circumvent mode collapse. VS prompts the model to verbalize a probability distribution over a set of responses (e.g., "Generate 5 jokes about coffee and their corresponding probabilities"). Comprehensive experiments show that VS significantly improves performance across creative writing (poems, stories, jokes), dialogue simulation, open-ended QA, and synthetic data generation, without sacrificing factual accuracy and safety. For instance, in creative writing, VS increases diversity by 1.6-2.1× over direct prompting. We further observe an emergent trend that more capable models benefit more from VS. In sum, our work provides a new data-centric perspective on mode collapse and a practical inference-time remedy that helps unlock pre-trained generative diversity.

Problem: ***Typicality Bias*** **Causes Mode Collapse**

Solution: Verbalized Sampling (VS) Mitigates Mode Collapse

Different prompts collapse to different modes:

Michal Malewicz

The End of Dashboards and Design Systems

Design is becoming quietly human again.

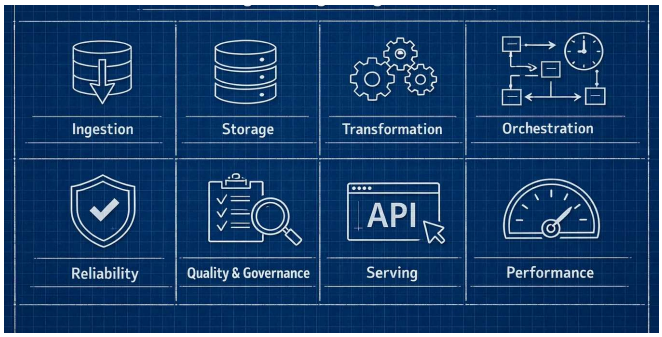
Nov 26, 2025 5.9K 229

In Generative AI by Adham Khaled

Stanford Just Killed Prompt Engineering With 8 Words (And I...

ChatGPT keeps giving you the same boring response? This new technique unlocks 2×...

Oct 19, 2025 24K 646

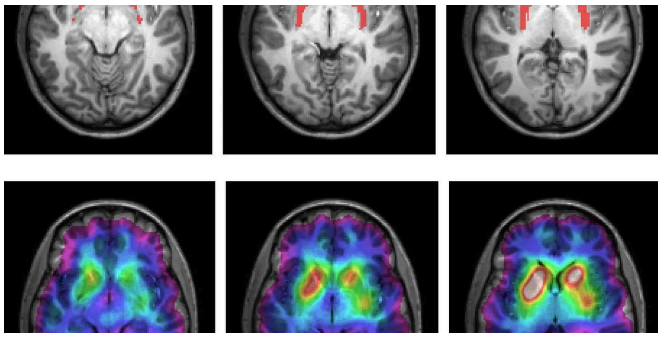


In AWS in Plain English by Khushbu Shah

Data Engineering Design Patterns You Must Learn in 2026

These are the 8 data engineering design patterns every modern data stack is built on....

Jan 5 1.1K 26



In Write A Catalyst by Dr. Patricia Schmidt

As a Neuroscientist, I Quit These 5 Morning Habits That Destroy You...

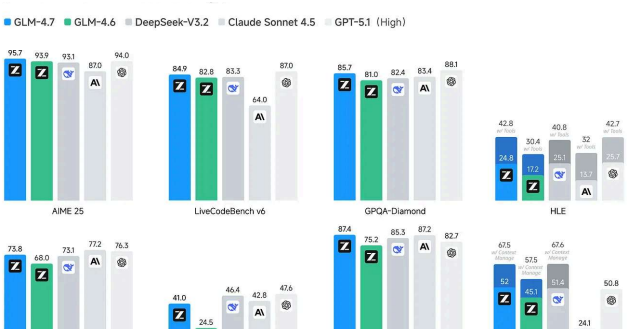
Most people do #1 within 10 minutes of waking (and it sabotages your entire day)

Jan 14 33K 620

In Activated Thinker by Shane Collins

Why the Smartest People in Tech Are Quietly Panicking Right Now

The water is rising fast, and your free version of ChatGPT is hiding the terrifying,...



Agent Native

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code...

Feb 13 11.1K 486

Jan 20 1.5K 42

See more recommendations